SYNKOPIA LAB · TECHNIK-NOTIZ

So funktioniert das Dub Techno Labor

Eine einzige HTML-Datei mit rund 50 KB, ohne Bibliotheken, ohne Audiodateien und ohne Server. Alles, was man sieht und hört, berechnet der Browser selbst.

01 Tech-Stack

BAUSTEIN	AUFGABE	UMFANG
HTML	Gerüst: Tabs, Panels, Regler, leere Container für Raster und Lernstufen	ca. 100 Zeilen
CSS	Aussehen: Farben als Variablen, Hell/Dunkel, Grid-Layout, Handy-Ansicht	ca. 150 Zeilen
JavaScript	Logik: Zustand, Sequencer, Lernstufen, Prüfen, Tipp, Zeitleiste, Quiz	ca. 440 Zeilen
Web Audio API	Klangerzeugung und Effekte, im Browser eingebaut	Teil des JS
Canvas 2D	Wellenform-Anzeige	Teil des JS
localStorage	speichert Song, Stufe und Fortschritt im Browser	Teil des JS
Google Fonts	drei Schriften: Bricolage Grotesque, Instrument Sans, IBM Plex Mono	einzige externe Quelle

Kein Framework (React, Vue), kein Build-Schritt, keine Installation. Die Datei läuft per Doppelklick oder als veröffentlichte Seite auf claude.ai.

02 Aufbau der Datei

```
<style>   Design-Tokens (Farben, Schriften) → Layout → Komponenten (Buttons, Raster, Regler)
<div>     drei Bereiche: Klang-Labor · Zeitleiste · Quiz & Lernkarten
<script>  Speicher → Tabs → Zustand S → Lernstufen → Audio-Motor → Sequencer
          → Raster-UI → Regler-UI → Prüfen/Tipp → Wellenform → Zeitleiste → Quiz
```

Der gesamte Code steckt in einer Funktion, die sich sofort selbst ausführt. So bleiben alle Variablen privat und stören nichts anderes auf der Seite.

03 Styling

Design-Tokens. Jede Farbe ist eine CSS-Variable, die an einer Stelle definiert wird. Für den Dunkelmodus werden nur die Variablen neu belegt, keine einzelnen Elemente.

```
:root { --accent:#a8650f; --echo:#2b7a8b; --
hint:#b5267e; }
@media (prefers-color-scheme: dark) {
  :root { --accent:#e2a64a; --echo:#62b3c4; …
}
}
.btn.primary { background: var(--accent); }
```

Layout. CSS Grid und Flexbox mit `gap`. Unter 820 px Breite rutschen die Panels untereinander. Die Überschrift skaliert mit `clamp()`.

Farbe trägt Bedeutung. Bernstein für Aktionen und den Akkord, Petrol für Echo und Wellenform, Magenta nur für Tipps, Grün für „richtig“.

Barrierearm. Sichtbarer Fokus-Rahmen für die Tastatur; Animationen entfallen bei „Bewegung reduzieren“.

04 Buttons und Regler

Echte `<button>`-Elemente. Sie funktionieren mit Maus, Touch und Tastatur. Der Zustand „an/aus“ steht im Attribut `aria-pressed`, und das CSS färbt danach. Screenreader lesen den Zustand dadurch automatisch vor.

```
.seq button[aria-pressed="true"]        { background: var(--ink); }   /* gesetzte Note */
.seq button.chord[aria-pressed="true"]  { background: var(--accent); }
.seq button.hint  { box-shadow: 0 0 0 2px var(--hint); }            /* Tipp-Umrandung */
```

Das Raster wird erzeugt, nicht geschrieben. Eine Schleife baut 4 Spuren × 16 Felder = 64 Buttons. Jeder Klick kippt einen Wert im Zustand um und speichert:

```
b.addEventListener("click", () => {
  S.pat[k][i] = !S.pat[k][i];               // Note an/aus im Zustand
  b.setAttribute("aria-pressed", S.pat[k][i]); // Aussehen folgt
  save();                                   // in localStorage
});
```

Regler sind native Schieberegler (`<input type="range">`). Alle hängen an derselben kleinen Funktion: Wert in den Zustand schreiben, Anzeige aktualisieren, an den Audio-Motor weitergeben.

Dub-Throw reagiert auf `pointerdown` / `pointerup`, also auf Drücken und Loslassen, damit man ihn wie einen echten Knopf halten kann.

05 Zustand und Datenfluss

Alles, was den Song ausmacht, steht in **einem** Objekt `S`: Tempo, Muster, Reglerwerte. Jede Eingabe läuft denselben Weg:



Die Lernstufen ändern den Zustand nicht. Sie zeigen nur Anleitung, Prüf-Ziele und Tipp-Ziele. „Prüfen“ vergleicht `S` mit diesen Zielen, und Regler gelten mit einer Toleranz als richtig, etwa ±10 %.

06 Klangerzeugung

Die Web Audio API ist ein Modular-Synthesizer im Browser: Man erzeugt Module (Oszillator, Filter, Verstärker, Echo, Hall) und verkabelt sie. Es gibt keine Samples, jeder Ton wird beim Abspielen berechnet.

INSTRUMENT	BAUWEISE
Kick	Sinus, Tonhöhe fällt in 0,13 s von 130 auf 42 Hz, klingt in 0,4 s aus
Hi-Hat	Rauschen durch Hochpass 7.500 Hz, 0,1 s kurz
Bass	Sinus auf C (65 Hz), 0,2 s
Akkord	Cm7 aus 8 Sägezahn-Oszillatoren (je Ton 2, um ±7 Cent verstimmt) → Tiefpassfilter
Rauschen	Schleife aus leisem Rauschen mit seltenen Knacksern, Bandpass 2.500 Hz

Signalweg. Kick, Hi-Hat und Bass bleiben trocken. Nur der Akkord geht über Abzweige (Sends) in Echo und Hall:



Space-Echo-Nachbau. Verzögerung (punktierte Achtel = 375 ms bei 120 BPM) → Tiefpass 2.200 Hz → Hochpass 180 Hz → zurück in die Verzögerung. Weil jede Wiederholung erneut durch die Filter läuft, wird sie dunkler, wie beim echten Tonband. Ein langsamer Oszillator (0,55 Hz) lässt die Echozeit um wenige Millisekunden schwanken, das ist das „Band-Leiern".

Hall. Eine berechnete Impulsantwort aus Rauschen, das über die Hall-Länge leiser wird. Der Faltungshall (`ConvolverNode`) legt diesen „Raum" über den Akkord.

07 Laufzeit: warum der Takt nicht stolpert

JavaScript-Zeitgeber sind ungenau, sie kommen oft ein paar Millisekunden zu spät. Der Sequencer plant deshalb voraus und überlässt das genaue Timing der Audio-Uhr:

```
setInterval(scheduler, 25);                 // alle 25 ms nachsehen …
function scheduler() {
  while (nextTime < ctx.currentTime + 0.12) { // … was in den nächsten 120 ms fällig ist
    playStep(step, nextTime);               // Note mit exakter Startzeit einplanen
    nextTime += 60 / S.bpm / 4;             // eine Sechzehntel weiter (125 ms bei 120 BPM)
    step = (step + 1) % 16;
  }
}
```

Die Audio-Uhr startet jede Note auf das Sample genau. Kommt JavaScript einmal zu spät, ist die Note längst eingeplant. Reglerbewegungen gleiten mit `setTargetAtTime()` in wenigen Millisekunden zum neuen Wert, statt zu springen. Deshalb knackst nichts.

08 Warum es so flüssig und schnell läuft

Audio läuft in einem eigenen Thread.
Der Browser rechnet den Klang getrennt von der Bedienoberfläche, mit hoher Priorität und in nativem Code. Klicks und Animationen bremsen den Sound nicht.

Nichts zu laden.
50 KB Text statt Megabytes an Samples und Bibliotheken. Die Seite ist sofort da.

Kein Framework.
Es gibt keine virtuelle Zwischenschicht. Der Code ändert nur genau das Element, das sich ändert, zum Beispiel ein Feld im Raster.

Kurzlebige Klänge.
Jede Note ist ein eigener Oszillator, der nach dem Abklingen stoppt und vom Browser aufgeräumt wird.

Zeichnen im Bildschirmtakt.
Die Wellenform wird mit `requestAnimationFrame` synchron zur Bildwiederholung gezeichnet, also 60-mal pro Sekunde und nie öfter als nötig.

Nebel-Effekt fast gratis.
Statt das Bild zu löschen, wird es halbtransparent übermalt. Das erzeugt die Spuren ohne zusätzliche Rechenarbeit.

09 Grenzen und Ausblick für Synkopia

- Synthese statt Samples: Die Kick ist weniger druckvoll als eine echte TR-909, und der Hall ist kein echter Federhall.
- Für die App mit vielen Stilen bietet sich **Tone.js** an. Es baut auf derselben Web Audio API auf und bringt fertige Synthesizer, Effekte und Timing mit.
- Ein Stil wird zu einer Datendatei (siehe `docs/blueprint-stilmodul.md`), der Klangmotor bleibt für alle Stile gleich.